

Comunicação entre aplicações

Laboratórios de Sistemas e Serviços

Mário Antunes

2026

Exercícios

Configuração

Antes de começar, vamos configurar o seu sistema com todas as ferramentas necessárias para estes exercícios.

Ferramentas de Sistema e Python

Primeiro, atualize as suas listas de pacotes e instale os utilitários principais: curl e wget para testar serviços web, e o gestor de pacotes do Python (pip) e o módulo de ambientes virtuais (venv).

1. Atualize as suas listas de pacotes

```
sudo apt update; sudo apt full-upgrade -y; \  
sudo apt autoremove -y; sudo apt autoclean
```

2. Instale ferramentas gerais, flatpak, e essenciais do Python

```
sudo apt install -y udisks2 curl wget \  
flatpak python3-pip python3-venv
```

3. Adicione o repositório Flathub

```
flatpak --user remote-add --if-not-exists \  
flathub https://flathub.org/repo/flathub.flatpakrepo
```

Boas Práticas de Python

Aqui vai encontrar uma lista de boas práticas para realizar projectos em Python3. Para cada projecto de Python, por favor, siga estes passos:

1. Crie um novo diretório para o projeto (ex: mkdir ex01 && cd ex01).

2. Crie um ambiente virtual isolado:

```
python3 -m venv venv
```

3. Ative o ambiente:

```
source venv/bin/activate
```

4. Crie um ficheiro requirements.txt (conforme especificado em cada exercício) e instale a partir dele:

```
pip install -r requirements.txt
```

5. Use o módulo logging em vez de print() para todas as suas mensagens de estado.

```
import logging  
logging.basicConfig(level=logging.INFO, format='%(message)s')  
logger = logging.getLogger(__name__)
```

```
logger.info("Esta é uma mensagem de informação.")
```

Aviso sobre a rede

Tipicamente, usará a rede Eduroam para aceder à internet durante as aulas. Para a maioria das atividades, isto é suficiente; no entanto, esta rede (gerida pela universidade) bloqueia a comunicação entre **equipamentos** dos estudantes. Tenha atenção, que sem estar ligado a outra rede, comunicação com outros alunos é muito difícil.

Exercício 1: Transferência de Ficheiros por UDP

Objetivo: Explorar o script `file_transfer.py` fornecido. Perceber como ele usa `asyncio` para criar um servidor persistente que pode lidar com múltiplos uploads de ficheiros de clientes.

Detalhes:

- **Servidor:** O servidor é persistente. Usa um `dict` para gerir transferências de ficheiros de diferentes clientes, usando como chave o seu IP e porta (`addr`).
- **Cliente:** O cliente envia primeiro os metadados do ficheiro (nome, tamanho), depois envia os pedaços (chunks) de dados, mostrando uma barra de progresso com `tqdm`.
- **Protocolo:** O script usa um protocolo simples baseado em nova linha (`newline`):
 - `START:<total_chunks>:<total_size>:<filename>`
 - `DATA:<chunk_num>:<data_chunk>`
 - `END`
- O servidor responde com `ACK_ALL` ou `ACK_FAIL`.

Instruções:

1. Crie um novo diretório `ex01` e entre nele `cd ex01`.
2. Descarregue o [código](#) da solução para este diretório.
3. Ative um `venv` e instale os requisitos:

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```
4. Crie um ficheiro para enviar, ex: `echo "Este é um ficheiro de teste UDP." > test.txt`.
5. **Execute o Servidor (Terminal 1):**

```
python file_transfer.py receive --port 9999
```
6. **Execute o Cliente (Terminal 2):**

```
python file_transfer.py send test.txt --host 127.0.0.1 --port 9999
```

Exercício 2: Jogo do Galo Remoto

Objetivo: Analisar o script `main.py` fornecido para ver como o `asyncio` pode ser integrado com uma biblioteca gráfica (GUI) como o Pygame para criar uma aplicação de rede.

Detalhes:

- **Menus GUI:** O script usa Pygame para desenhar todos os seus próprios menus. Não usa `argparse`.
- **Loop de Jogo Async:** O loop principal `while running:` é `async`. Ele cede o controlo ao event loop do `asyncio` ao chamar `await asyncio.sleep(1/FPS)`.
- **Rede:** O script usa `asyncio.start_server` (para o anfitrião/host) e `asyncio.open_connection` (para o cliente) para criar streams TCP fiáveis.
- **Tratamento de Erros:** As funções `run()` e `close_connection()` usam `try...finally` e tratam `asyncio.CancelledError` para garantir que a aplicação encerra de forma limpa.

Instruções:

1. Crie um novo diretório `ex02` e entre nele `cd ex02`.
2. Descarregue o [código](#) da solução para este diretório.
3. Ative um `venv` e instale os requisitos:

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

4. Execute o Anfitrião (Host - Jogador X):

```
python main.py
```

- Na GUI, clique em "Host Game" -> insira uma porta (ex: 8888) -> Pressione Enter.

5. Execute o Cliente (Jogador O):

```
python main.py
```

- Na GUI, clique em "Join Game" -> insira o IP do anfitrião (127.0.0.1 se for na mesma máquina) -> Pressione Enter -> insira a porta (8888) -> Pressione Enter.

Exercício 3: Serviço de Cache com FastAPI

Objetivo: Executar e testar o script `main.py` fornecido para entender como construir um endpoint de API de alta performance com cache.

Detalhes:

- **Endpoint:** O script fornece um endpoint GET `/ip/{ip_address}`.
- **Cache:** Usa um ficheiro local `ip_cache.json`.
- **Lógica:** Verifica o timestamp de uma entrada em cache contra um `CACHE_DEADLINE_SECONDS`.
- **API Externa:** Se a cache estiver desatualizada (stale) ou em falta, usa a biblioteca `requests` para obter dados ao vivo.

Instruções:

1. Crie um novo diretório `ex03` e entre nele `cd ex03`.
2. Descarregue o [código](#) da solução para este diretório.
3. Ative um venv e instale os requisitos:

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

4. Execute o Servidor:

```
uvicorn main:app --reload
```

5. Teste o Serviço (num novo terminal):

- **Teste 1 (Falha na Cache - Miss):**

```
# IP Privado (tem de falhar)
curl http://127.0.0.1:8000/ip/192.168.132.132
```

```
# DNS Google
curl http://127.0.0.1:8000/ip/8.8.8.8
```

```
# IP Público da MEO
curl http://127.0.0.1:8000/ip/144.64.3.83
```

```
# UA
curl http://127.0.0.1:8000/ip/193.137.169.135
```

```
# IP Estático de São Tomé
curl http://127.0.0.1:8000/ip/197.159.166.30
```

(Verifique os logs do servidor; deve dizer "Querying external API".)

- **Teste 2 (Sucesso na Cache - Hit):**

```
# IP Privado (tem de falhar)
curl http://127.0.0.1:8000/ip/192.168.132.132

# DNS Google
curl http://127.0.0.1:8000/ip/8.8.8.8

# IP Público da MEO
curl http://127.0.0.1:8000/ip/144.64.3.83

# UA
curl http://127.0.0.1:8000/ip/193.137.169.135

# IP Estático de São Tomé
curl http://127.0.0.1:8000/ip/197.159.166.30

(Verifique os logs do servidor; deve dizer "Returning cached data".)
```

Exercício 4: Chat Pub/Sub

Objetivo: Usar Docker para executar um broker MQTT e ligar-se a ele com um cliente puramente JavaScript para criar uma aplicação de chat "serverless".

Detalhes:

- **Sem Servidor Python:** Você não vai escrever *nenhum* código de servidor. O broker Mosquitto é o servidor.
- **Broker:** O ficheiro `docker-compose.yml` inicia o Mosquitto e carrega o `mosquitto.conf`.
- **Configuração:** O ficheiro `.conf` ativa o acesso anónimo e abre a porta 9001 para **MQTT-sobre-WebSockets**.
- **Cliente:** O ficheiro `chat_client.html` usa a biblioteca **MQTT.js** (carregada de um CDN) para se ligar a `ws://localhost:9001`. Implementa um chat Pub/Sub.

Instruções:

1. Crie um novo diretório `ex04` e entre nele `cd ex04`.
2. Descarregue o [código](#) para o mesmo diretório.
3. **Inicie o Broker:**

```
docker compose up -d
```
4. **Teste o Cliente:**
 - Abra `http://localhost:8080/` no seu navegador web.
 - Abra `http://localhost:8080/` num *segundo* separador ou janela do navegador.
 - Insira nomes de utilizador diferentes e ligue-se. As mensagens enviadas numa janela devem aparecer na outra.
 - Pode usar a rede TheOffice para conversar com outros estudantes.

Exercício Bónus: O Clássico Servidor de Eco (Echo Server)

Objetivo: Escrever um Servidor de Eco (Echo Server) simples em Python usando o módulo `socket` incorporado. Este é o "Olá, Mundo!" da programação em rede.

Tarefa: Este é o único exercício onde **tem de escrever o código você mesmo**.

Crie um único script Python `echo_server.py`. O script deve ser capaz de correr num de dois modos usando `argparse`:

1. `python echo_server.py tcp --port <num>`
2. `python echo_server.py udp --port <num>`

Requisitos:

- **Modo TCP:** O servidor deve escutar na porta indicada, aceitar uma conexão de cliente, e `recv` (receber) dados do cliente. Deve então `sendall` (enviar tudo) os *exatos mesmos dados* de volta. Deve lidar graciosamente com clientes que se desligam.

- **Modo UDP:** O servidor deve fazer bind à porta indicada, recvfrom (receber de) um datagrama, e sendto (enviar para) os *exatos mesmos dados* de volta para o endereço de onde vieram.
- Tem de escrever este código de raiz. **Não use asyncio para este exercício.**
- Teste o seu servidor TCP com netcat: nc 127.0.0.1 <porta>.
- Teste o seu servidor UDP com netcat: nc -u 127.0.0.1 <porta>.

Documentação Útil:

- **Módulo socket do Python:** <https://docs.python.org/3/library/socket.html>
- **Guia HOWTO de Programação de Sockets em Python:** <https://docs.python.org/3/howto/sockets.html>